Lecture 1 - Linear Regression, Classification and logistic regression, Generalized Linear Models

---

Supervised Learning

Giving a dataset of living areas and prices of 47 houses (equivalent to 47 samples), so how can we predict the prices of other houses? $\rightarrow$ **A function of the size of their living areas?**

First, we have the input variables $x^{(i)}$, called **features**, and $y^{(i)}$ denote the output or **target** variable that we are trying to predict the price. A pair $(x^{(i)}, y^{(i)})$ is called a **training example**.

When the target variable is continuous, we call that learning problem a **regression** problem. Or when we want to predict if a dwelling is a house or an apartment, we call it a **classification** problem.

1. Linear Regression

Living area (feet^2)	#bedrooms	Price (1000\$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
$\vdots$	$\vdots$	$\vdots$

We added another feature to that dataset, so here we have a two-dimensional vectors in $\mathbb{R}^2$. Suppose we decide to approximate y as a linear function of x:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Here, the θ_i are the **parameters** or **weights** parameterizing the space of linear functions mapping from $\mathcal{X}$ to $\mathcal{Y}$.

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x$$

To make $h(x)$ close to y, we define a function that measures, for each value of the θ , **cost function**

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

it is also called **ordinary least squares** regression model.

1.1 LMS algorithm

To converge to a value θ that minimizes $J(\theta)$, let's consider the **gradient descent** algorithm, which starts with some initial θ and repeatedly performs the update:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Suppose we have only one training sample (x, y), so

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2 \\
&= (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} (\sum_{i=0}^{n} \theta_i x_i - y) \\
&= (h_\theta(x) - y) \cdot x_j
\end{aligned}$$

This gives rise to

$$\theta_j = \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

The rule is called **least mean squares** or **Widrow-Hoff** learning rule. You can see that when our prediction nearly matches the actual value of $y^{(i)}$, then the final value of θ_j is the final one.

$$\begin{aligned}
&\text{Repeat until convergence } \{ \\
&\qquad \theta_j := \theta_j + \alpha \textstyle\sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)} \qquad (\text{for every } j). \\
&\}
\end{aligned}$$

The above algorithm is called **batch gradient descent** because it looks at every example in the entire training on every step.

$$\begin{aligned}
&\text{Loop } \{ \\
&\qquad \text{for i=1 to m, } \{ \\
&\qquad\qquad \theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)} \qquad (\text{for every } j). \\
&\qquad \} \\
&\}
\end{aligned}$$

And here, we repeatedly run through the training set, and each time we encounter a training example, we update the parameters according to the gradient of the error with respect to that the single training example only. This algorithm is called **stochastic gradient descent**

1.2 Normal Equation

This section will discuss a second way of minimizing J . First, we need to list out some notation related to matrix.

1.2.1 Matrix derivatives

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}$$

For example, suppose $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ and a function $f : \mathbb{R}^{2\times 2} \to \mathbb{R}$ is given by

$$f(A) = \frac{3}{2}A_{11} + 5A_{12}^2 + A_{21}A_{22}$$

We then have,

$$\nabla_A f(A) = \begin{bmatrix} \frac{3}{2} & 10A_{12} \\ A_{22} & A_{21} \end{bmatrix}$$

We also introduce the **trace** operator, written "tr".

$$\mathrm{tr} A = \sum_{i=1}^{n} A_{ii}$$

$$\mathrm{tr}\, A = \mathrm{tr}\, A^T$$

$$\mathrm{tr}(A+B) = \mathrm{tr}\, A + \mathrm{tr}\, B$$

$$\mathrm{tr}\, aA = a\, \mathrm{tr}\, A$$

$$\mathrm{tr}\, ABC = \mathrm{tr}\, CAB = \mathrm{tr}\, BCA$$

$$\mathrm{tr}\, ABCD = \mathrm{tr}\, DABC = \mathrm{tr}\, CDAB = \mathrm{tr}\, BCDA$$

$$\nabla_A \mathrm{tr}\, AB = B^T \tag{1}$$

$$\nabla_{A^T} f(A) = (\nabla_A f(A))^T \tag{2}$$

$$\nabla_A \mathrm{tr}\, ABA^TC = CAB + C^TAB^T \tag{3}$$

$$\nabla_A |A| = |A|(A^{-1})^T \tag{4}$$

1.2.2 Least squares revisited

We present the feature variables and target variable in form of matrices.

$$X = \begin{bmatrix} — (x^{(1)})^T — \\ — (x^{(2)})^T — \\ \vdots \\ — (x^{(m)})^T — \end{bmatrix}, \quad \vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}.$$

Then

$$\frac{1}{2}(X\theta - \vec{y})^T(X\theta - \vec{y}) = \frac{1}{2}\sum_{i=1}^{n}(h_\theta(x^{(i)}) - y^{(i)})^2 = J(\theta)$$

Taking the derivatives of $J(\theta)$

$$\nabla_\theta J(\theta) = X^T X\theta - X^T \vec{y}$$

To minimize J, we set its derivatives to zero, obtain the **normal equations**

$$X^T X\theta = X^T \vec{y}$$

Thus, the value of θ that minimizes $J(\theta)$ is given in closed form by the equation

$$\theta = (X^T X)^{-1} X^T \vec{y}$$

Key Notes

Joint Probability under IID Assumption

If samples are **independent and identically distributed (IID)**, the joint probability equals the product of individual probabilities:

$$P(A_1 \text{ and } A_2 \text{ and } \cdots \text{ and } A_m) = \prod_{i=1}^{m} P(A_i)$$

Why Log-Likelihood?

The Likelihood function is a product of many probabilities (values < 1), which causes two problems:

1. **Numerical underflow**: multiplying many small numbers drives the result toward 0
2. **Hard to differentiate**: optimizing a product is complex

Taking the log converts the product into a sum:

$$\log \prod_{i=1}^{m} p_i = \sum_{i=1}^{m} \log p_i$$

Since $\log$ is a monotonically increasing function, maximizing $L(\theta)$ and maximizing $\log L(\theta)$ yield the same optimal θ.

1.2.3 Locally Weighted Linear Regression (LWR)

Problem with Standard Linear Regression
Standard linear regression fits **one global line** to all data. If the data is not truly linear, the fit will be poor (underfitting). Adding features (x^2, x^3...) can help, but choosing the right features is difficult.

LWR Idea
Instead of fitting a global model, LWR says: **when predicting at point x, only care about nearby data points**.

Standard Linear Regression: minimize $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$ — all data points treated **equally**.

LWR: minimize $\sum_i w^{(i)}(y^{(i)} - \theta^T x^{(i)})^2$ — each data point has a different **weight** $w^{(i)}$.

How Weights Work

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

- $x^{(i)}$ **close** to query point x → $|x^{(i)} - x|$ small → $w^{(i)} \approx 1$ → **high influence**
- $x^{(i)}$ **far** from query point x → $|x^{(i)} - x|$ large → $w^{(i)} \approx 0$ → **nearly ignored**

τ (**bandwidth**) controls how far "close" means:

- τ small → only very nearby points matter
- τ large → wider attention (approaches standard linear regression)

Parametric vs Non-parametric

	Linear Regression (Parametric)	**LWR (Non-parametric)**
After training	Discard data, keep θ	Must keep **entire** training set
Prediction	Use fixed θ	Recompute θ for each query
Parameters	Fixed number	Grows with data size

Summary
LWR avoids the feature selection problem by fitting a **local linear model** around each query point using only nearby data, at the cost of higher computation since it must retain all training data and refit for every prediction.

2. Classification and logistic regression

2.1 Logistic Regression

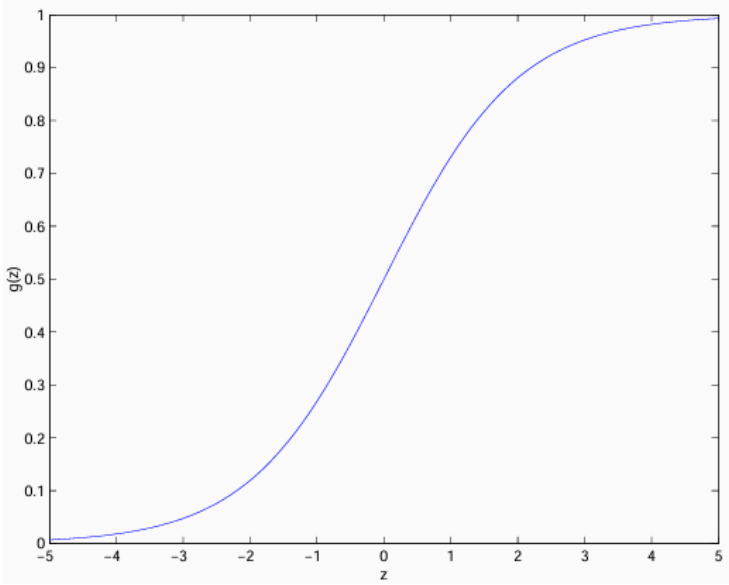
Linear Regression is no longer suitable for classification problem, where y is discrete-valued $y \in \{0, 1\}$.Let's change tthe form of our hypothesis $h_\theta(x)$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

where

$$g(z) = \frac{1}{1 + e^{-z}}$$

is called the **logistic function** or the **sigmoid function**.



The derivative of the sigmoid function is then,

$$g'(z) = g(z)(1 - g(z))$$

Let us assume that

$$P(y = 1|x; \theta) = h_\theta(x)$$

$$P(y = 0|x; \theta) = 1 - h_\theta(x)$$

$$P(y|x; \theta) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}$$

Assuming that the m training examples were generated independently, we can write down the likelihood as

$$L(\theta) = \prod_{i=1}^{m} \left(h_\theta(x^{(i)})\right)^{y^{(i)}} \left(1 - h_\theta(x^{(i)})\right)^{1-y^{(i)}}$$

As before, it will be easier to maximize the log likelihood

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{m} y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

Contrasting with cost function $J(\theta)$, the log likelihood is written in $\theta := \theta + \alpha \nabla_\theta \ell(\theta)$, because we are maximizing, not minimizing.

$$\frac{\partial}{\partial \theta_j} \ell(\theta) = (y - h_\theta(x)) x_j$$

Therefore,

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

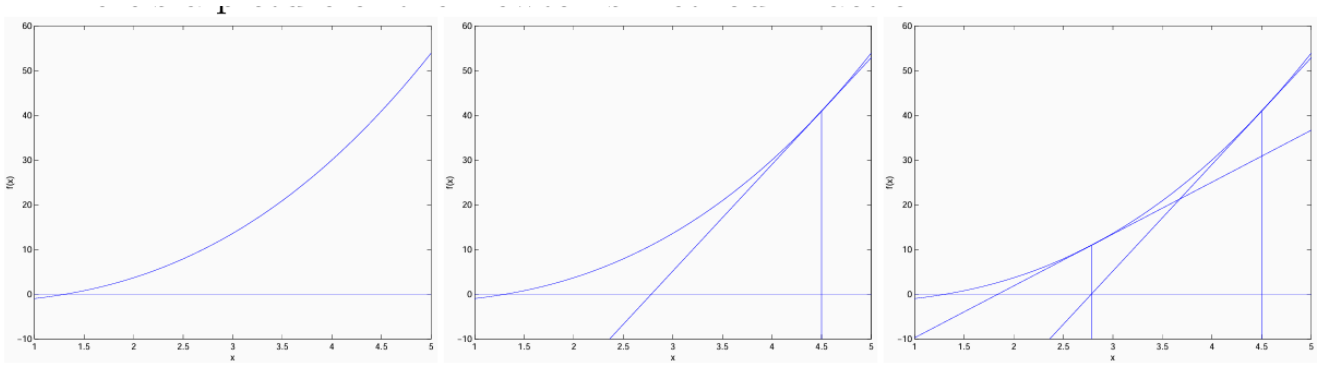
2.2 Another algorithm for maximizing $\ell(\theta)$

$$L(\theta') = f(\theta) + f'(\theta)(\theta' - \theta)$$

This is a **linear function of** θ' , $f'(\theta)$ is the slope.

Newton's method finds the zero of f by repeatedly applying a linear (first-order Taylor) approximation at the current point:

$$\theta := \theta - \frac{f(\theta)}{f'(\theta)}$$



To **optimize** ℓ, set $f := \ell'$ — since extrema occur where $\ell'(\theta) = 0$. The update rule becomes:

$$\theta := \theta - \frac{\ell'(\theta)}{\ell''(\theta)}$$

When $\theta \in \mathbb{R}^n$, the scalar second derivative $\ell''(\theta)$ generalizes to the **Hessian matrix** $H \in \mathbb{R}^{n \times n}$, whose (i, j)-entry captures the second-order interaction between parameters θ_i and θ_j:

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}$$

The update rule becomes:

$$\theta := \theta - H^{-1} \nabla_\theta \ell(\theta)$$

Why H^{-1} instead of H? In the scalar case, dividing by ℓ'' scales the gradient by $\frac{1}{\ell''}$. In the vector case, division does not exist for matrices — multiplying by H^{-1} is the exact matrix analog of dividing by a scalar. Concretely, H^{-1} rescales and rotates the gradient vector $\nabla_\theta \ell$ according to the curvature in every direction, so the step size is automatically adjusted per dimension without needing a learning rate α.

3. Generalized Linear Models

3.1 The Exponential Family

A class of distributions is in the exponential family if it can be written in the form:

$$p(y, \eta) = b(y) \exp(\eta^T T(y) - a(\eta))$$

where η is called **natural parameter** or **canonical parameter** of the distribution; $T(y)$ is the **sufficient statistic**; and $a(\eta)$ is the **log partition function**. The quantity $e^{-a(\eta)}$ plays as normalization constant, in order to make sure that the distribution $p(y, \eta)$ sums/integrates over y to 1.

We start with an example of Bernoulli distribution to show that it is an exponential family distribution. The Bernoulli distribution with mean ϕ, written Bernoulli(ϕ) , specifies a distribution over $y \in \{0, 1\}$, so that

$$p(y = 1|\phi) = \phi$$
$$p(y = 0|\phi) = 1 - \phi$$

Then

$$\begin{aligned} p(y, \phi) &= \phi^y (1-\phi)^{1-y} \\ &= \exp^{\log(\phi^y (1-\phi)^{1-y})} \\ &= \exp^{\log(\phi^y) + \log((1-\phi)^{1-y})} \\ &= \exp^{y \log(\phi) + (1-y) log(1-\phi)} \\ &= \exp^{y\left(\log(\frac{\phi}{1-\phi})\right) + \log(1-\phi)} \end{aligned}$$

So the natural parameter $\eta = \log(\phi/(1-\phi))$, $T(y) = y$, $a(\eta) = -\log(1-\phi)$, $b(y) = 1$

Let's move to consider Gaussian distribution.

$$p(y; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{(y-\mu)^2}{2\sigma^2})$$

Set $\sigma^2 = 1$ then

$$\begin{aligned} p(y; \mu) &= \frac{1}{\sqrt{2\pi}} \exp(-\frac{(y-\mu)^2}{2}) \\ &= \frac{1}{\sqrt{2\pi}} \exp(-\frac{1}{2}y^2 + y\mu - \frac{1}{2}\mu^2) \\ &= \frac{1}{\sqrt{2\pi}} \exp(-\frac{1}{2}y^2) \cdot \exp(y\mu - \frac{1}{2}\mu^2) \end{aligned}$$

Then we can see that the Gaussian is in the exponential family, with

$$\begin{aligned} \eta &= \mu \\ T(y) &= y \\ a(\eta) &= \mu^2/2 \\ &= \eta^2/2 \\ b(y) &= (1/\sqrt{2\pi}) \exp(-y^2/2). \end{aligned}$$

3.2 Constructing GLMs

Context

So far, we have seen two examples:

- Linear regression: $y|x; \theta \sim \mathcal{N}(\mu, \sigma^2)$
- Logistic regression: $y|x; \theta \sim \text{Bernoulli}(\phi)$

The question is: is there a unified way to construct models for any distribution?

GLMs are the answer.

Three Assumptions of GLMs

To construct a GLM, we make three assumptions:
Assumption 1: $y|x;\theta \sim \text{ExponentialFamily}(\eta)$
The distribution of y given x belongs to the exponential family with natural parameter η.
Assumption 2: $h(x) = E[y|x]$
The hypothesis function must predict the **expected value** of y. That is, $h_\theta(x) = E[T(y)|x]$, and since typically $T(y) = y$, we have $h_\theta(x) = E[y|x]$.
Assumption 3: $\eta = \theta^T x$
The natural parameter η is a linear function of the input x. This is a **design choice** — not derived from mathematics, but a deliberate modeling decision.

How It Works

These three assumptions create a **recipe** for deriving a model:

$$x \xrightarrow{\text{Assumption 3}} \eta = \theta^T x \xrightarrow{\text{Assumption 1}} \text{distribution} \xrightarrow{\text{Assumption 2}} h(x) = E[y|x]$$

Specifically: for each distribution in the exponential family, there exists a function g such that:

$$E[y;\eta] = g(\eta)$$

This function g is called the **canonical response function**. Its inverse g^{-1} is called the **canonical link function**.

Application 1: Ordinary Least Squares

Choose $y|x;\theta \sim \mathcal{N}(\mu, \sigma^2)$.
We already know that the Gaussian has $\eta = \mu$. Applying the three assumptions:

$$h_\theta(x) = E[y|x;\theta] = \mu = \eta = \theta^T x$$

The canonical response function is the **identity function** — $g(\eta) = \eta$.
Result: linear regression.

Application 2: Logistic Regression

Choose $y|x;\theta \sim \text{Bernoulli}(\phi)$.
We already know that the Bernoulli has $\eta = \log \frac{\phi}{1-\phi}$, which gives $\phi = \frac{1}{1+e^{-\eta}}$.
Applying the three assumptions:

$$h_\theta(x) = E[y|x;\theta] = \phi = \frac{1}{1+e^{-\eta}} = \frac{1}{1+e^{-\theta^T x}}$$

The canonical response function is the **sigmoid function**.
Result: logistic regression — the sigmoid arises not arbitrarily, but as a necessary

consequence of choosing the Bernoulli distribution.

Application 3: Softmax Regression

When $y \in \{1, 2, \ldots, k\}$ — multiclass classification. Choose the **multinomial distribution**. The multinomial has $k-1$ parameters $\phi_1, \ldots, \phi_{k-1}$ (since $\sum \phi_i = 1$, ϕ_k is determined by the rest).
The natural parameter is a vector:

$$\eta_i = \log \frac{\phi_i}{\phi_k}, \quad i = 1, \ldots, k-1$$

Inverting to find the response function:

$$e^{\eta_i} = \frac{\phi_i}{\phi_k} \implies \phi_i = \phi_k e^{\eta_i}$$

Since $\sum_{i=1}^{k} \phi_i = 1$:

$$\phi_k \sum_{i=1}^{k} e^{\eta_i} = 1 \implies \phi_k = \frac{1}{\sum_{j=1}^{k} e^{\eta_j}}$$

Therefore:

$$\phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^{k} e^{\eta_j}}$$

This is the **softmax function**. With $\eta_i = \theta_i^T x$:

$$p(y = i | x; \theta) = \frac{e^{\theta_i^T x}}{\sum_{j=1}^{k} e^{\theta_j^T x}}$$

Summary

Model	Distribution	Response Function
Linear regression	Gaussian	Identity: $g(\eta) = \eta$
Logistic regression	Bernoulli	Sigmoid: $g(\eta) = \frac{1}{1+e^{-\eta}}$
Softmax regression	Multinomial	Softmax: $g(\eta)_i = \frac{e^{\eta_i}}{\sum_j e^{\eta_j}}$

Core insight: GLMs are not three separate models. They are **one unified framework** — changing the distribution of y automatically determines the response function. Assumption 3 ($\eta = \theta^T x$) is the only arbitrary element; the other two assumptions have clear justification.